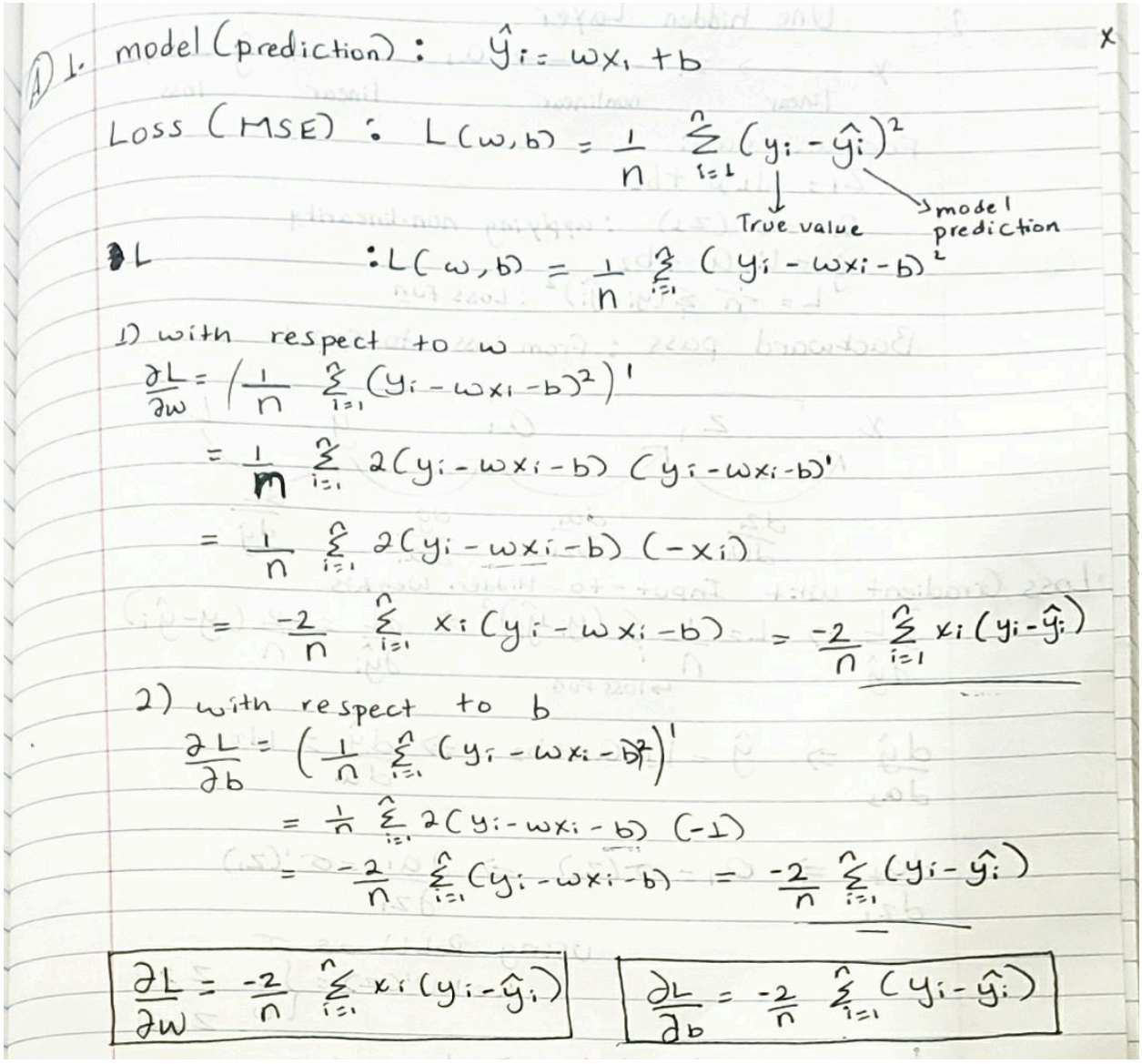
Deriving and Implementing Gradient-Based Learning, From Linear Regression to a One-Hidden-Layer Neural Network

Derivation

Minimizing error in model training is done by finding the lowest point of the error function. To find the minimum point we use derivatives.

1. Linear Regression Derivations



2. One-Hidden-Layer Neural Network Derivations

2. One hidden Layer

$x \xrightarrow[\text{linear}]{} z_1 \xrightarrow[\text{nonlinear}]{} a_1 \xrightarrow[\text{linear}]{} \hat{y} \xrightarrow[\text{loss}]{} L$

Forward pass:

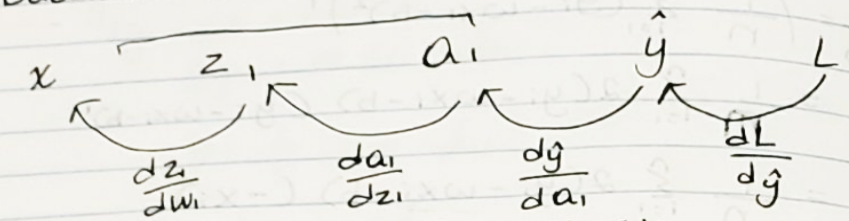
$z_1 = W_1 x + b_1$

$a_1 = \sigma(z_1)$: applying non-linearity

$\hat{y} = W_2 a_1 + b_2$

$L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$: Loss fun

Backward pass : From loss to input



Loss Gradient wrt Input-to-Hidden Weights

$\frac{\partial L}{\partial \hat{y}} \Rightarrow L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2 \Rightarrow \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{n}(y - \hat{y}_i)$

↳ loss Fun

$\frac{d\hat{y}}{da_1} \Rightarrow \hat{y} = W_2 a_1 + b_2 \Rightarrow \frac{d\hat{y}}{da_1} = W_2$

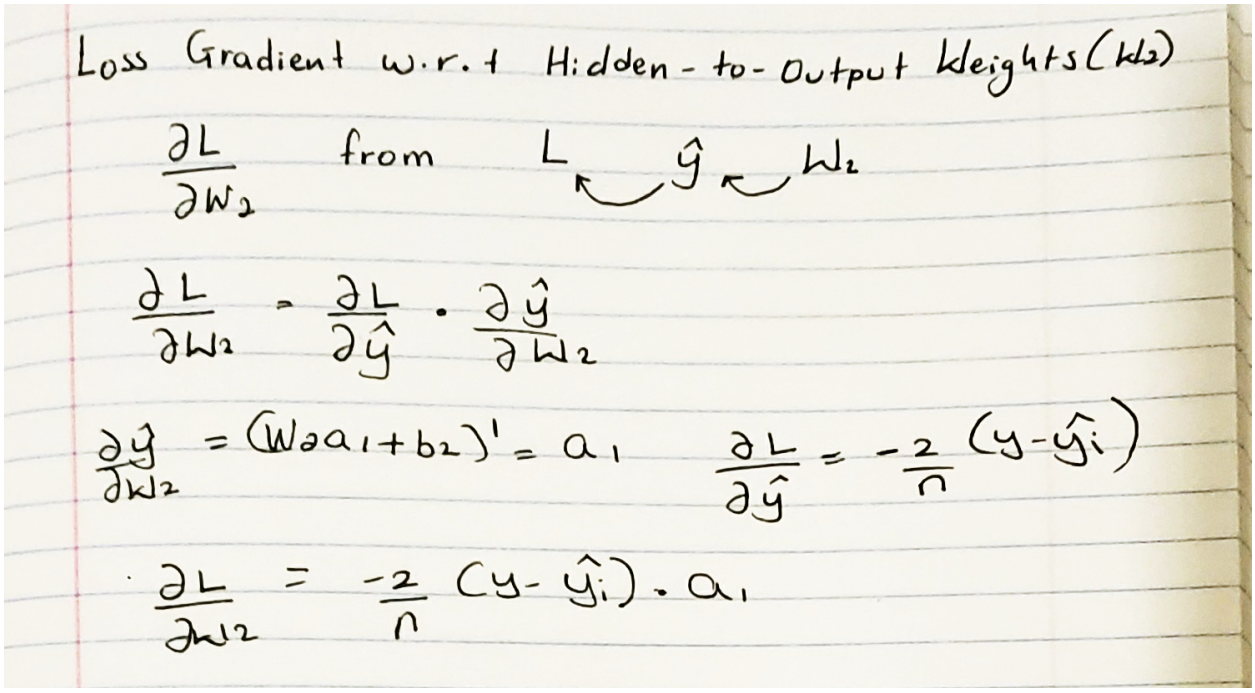
$\frac{da_1}{dz_1} \Rightarrow a_1 = \sigma(z_1) \Rightarrow \frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$

using ReLU as σ

$$ReLU'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

$\frac{dz_1}{dW_1} \Rightarrow z_1 = W_1 x + b_1 \Rightarrow \frac{dz_1}{dW_1} = x$

$$\frac{dL}{dW_1} = \frac{dL}{d\hat{y}} \times \frac{d\hat{y}}{da_1} \times \frac{da_1}{dz_1} \times \frac{dz_1}{dW_1} = -\frac{2}{n}(\hat{y} - \hat{y}) \cdot W_2 \cdot \sigma'(z_1) \cdot x$$



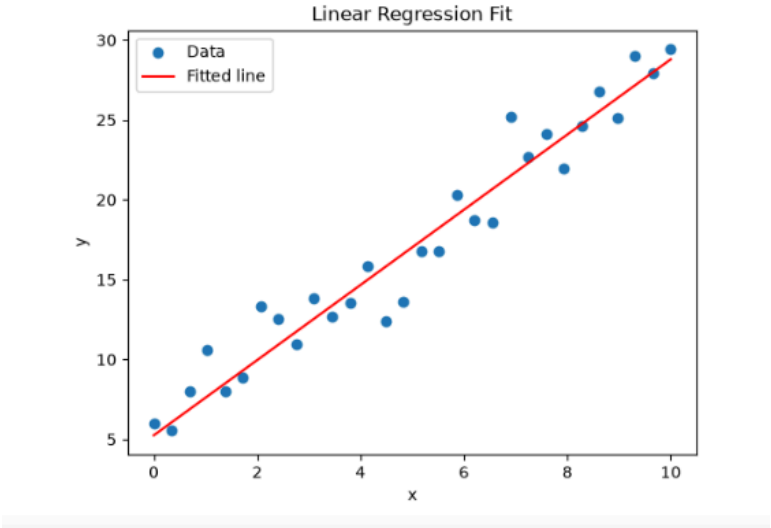
3. why reverse-mode differentiation is more efficient than forward-mode for this network.
    - The network has many parameters (all entries of W1, W2, and the biases) but only one scalar output: the loss L.
    - **Forward-mode differentiation** computes how one parameter affects the output by propagating derivatives from that parameter forward through each node (z1 → a1 → ŷ → L). To get gradients for *every* parameter, this process must be repeated separately for each one, so the cost scales with the number of parameters.
    - **Reverse-mode differentiation** starts at the loss and propagates derivatives backward (∂L/∂ŷ → ∂L/∂a1 → ∂L/∂z1). Each intermediate derivative is computed once and reused to obtain the gradient for every parameter that feeds into that node.
    - Since the network has far more parameters than outputs, reverse-mode computes the full gradient in a single backward pass, while forward-mode would require one pass per parameter. This makes reverse-mode far more computationally efficient for networks like this one, and is the reason backpropagation (reverse-mode AD) is the standard method for training neural networks.

      Simply put:
    - Forward-mode: you pick one weight, track its effect all the way to L → repeat this separately for every single weight.
    - Reverse-mode: you start from L, compute ∂L/∂ŷ once, then ∂L/∂a1 once, then ∂L/∂z1 once — and these reused values instantly give you gradients for *all* weights feeding into that node

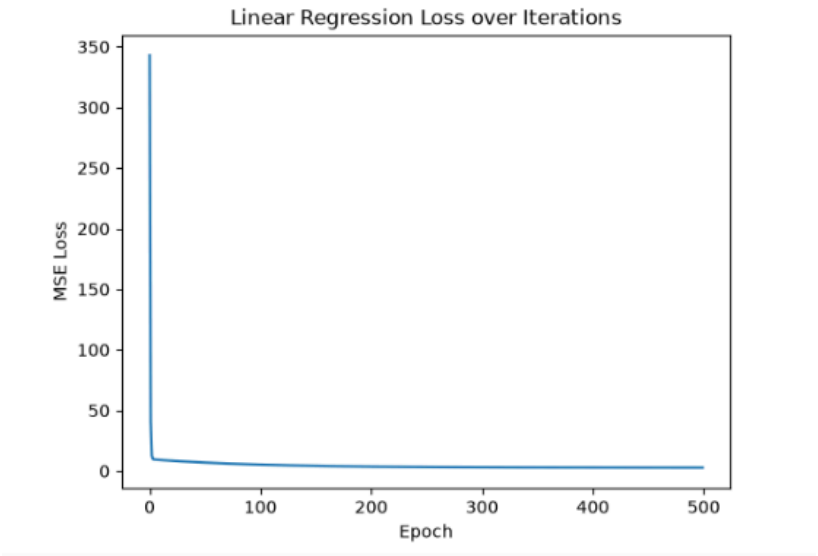
Backpropagation computes the gradient of the loss with respect to every parameter by applying the chain rule backward from the output to the input, reusing intermediate derivatives (∂L/∂ŷ, ∂L/∂a1, ∂L/∂z1) at each stage rather than recomputing them, which is precisely the efficiency advantage of reverse-mode differentiation discussed in Part A

Results and Observations

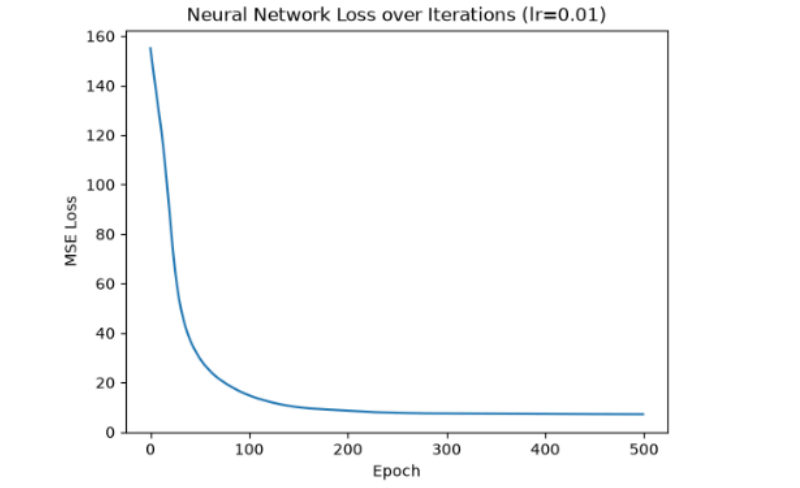
Linear Regression visualization: The fitted line closely tracks the data trend, confirming the gradient descent successfully minimized the loss and recovered parameters close to the true generating values (m≈2.5, b≈5)



Linear Regression Loss over Iterations: Loss dropped sharply within the first few epochs and plateaued near zero, indicating rapid convergence typical of a simple convex loss surface.



Neural Network Loss over Iterations (lr=0.01): The neural network's loss decreases smoothly but converges more slowly than linear regression, reflecting the added complexity of learning a nonlinear relationship through multiple parameters.



Neural Network Loss for Different Learning Rates: Learning rate significantly affects convergence behavior. A small rate (0.001) converges too slowly to reach a low loss within 500 epochs. A large rate (0.1) causes unstable oscillations early in training and converges to a higher loss floor, likely due to overshooting the minimum. The moderate rate (0.01) achieves the fastest and most stable

convergence, reaching the lowest loss.

